

Datalink Access

Chapter 29: Packet Capture, Raw Access and UDP Checksum Case Study

Arsha J K

Department of Computer Science & Engineering
Viswajyothi College of Engineering and Technology

Learning Objectives

By the end of this section, students should be able to:

- ▶ Explain why applications need access to the datalink layer.
- ▶ Describe packet capture and the role of promiscuous mode.
- ▶ Distinguish BPF, DLPI, SOCK_PACKET and PF_PACKET.
- ▶ Explain filtering, snapshot length, buffering and timestamps.
- ▶ Describe how **libpcap** provides implementation-independent packet capture.
- ▶ Explain how **libnet** simplifies packet creation and injection.
- ▶ Trace the UDP checksum case study from DNS query generation to captured reply.
- ▶ Explain raw sockets, IP_HDRINCL, packet filters, timeouts and checksum inspection.

[VISUAL ANCHOR: Chapter roadmap: datalink access → capture → raw packet creation → UDP checksum test]

29.1 Why Provide Datalink-Layer Access?

Packet Observation

- ▶ Applications can watch packets received by the datalink layer.
- ▶ This allows programs such as **tcpdump** to run on ordinary computer systems.
- ▶ With promiscuous mode, an interface can receive packets on the local cable beyond those addressed to the host.

Packet Generation

- ▶ Some programs can operate as normal applications rather than kernel components.
- ▶ Example: a Unix RARP server can read RARP requests from the datalink and write replies back.
- ▶ RARP requests are not IP datagrams.

Core Idea

Datalink access lets an application observe or create traffic below the normal IP interface.

[VISUAL ANCHOR: Application connected directly to the datalink layer]

Promiscuous Mode

- ▶ A network interface can be placed in promiscuous mode to watch packets passing on the local cable.
- ▶ Without it, normal reception is restricted to traffic intended for the host.

Why Switching Changes the Picture

- ▶ A switch normally forwards traffic only to the port containing the destination device(s).
- ▶ Traffic may be unicast, multicast or broadcast.
- ▶ To monitor other switch ports, the switch must be configured for **monitor mode / port mirroring**.
- ▶ The source notes that some devices that appear to be hubs may internally operate as switches.

[VISUAL ANCHOR: Switched LAN showing normal forwarding versus monitor/port-mirroring traffic]

Three Unix Datalink-Access Methods

Method	Role described in the chapter
BPF	BSD Packet Filter; packet capture and filtering
DLPI	SVR4 Datalink Provider Interface; protocol-independent datalink access
SOCK_PACKET	Original Linux datalink-receive interface
PF_PACKET	Newer Linux packet socket family with additional filtering/performance features

Portable Layer

libpcap hides OS-specific packet-capture mechanisms and provides implementation-independent access.

[VISUAL ANCHOR: BPF / DLPI / Linux packet sockets feeding a common libpcap interface]

29.2 BSD Packet Filter (BPF)

Architecture

- ▶ Berkeley-derived systems support BPF.
- ▶ A datalink driver calls BPF immediately after packet reception and as late as possible before transmission.
- ▶ This placement helps provide **accurate timestamps**.

Main Strength

- ▶ Each application opening a BPF device can load its own filter.
- ▶ BPF applies the filter to packets before delivering selected data to the application.

[VISUAL ANCHOR: Textbook Figure 29.1 – Packet capture using BPF]

Simple Filter

`udp or tcp`

- ▶ Selects UDP or TCP packets.

Header-Aware Filter

`tcp and port 80 and`

`tcp[13:1] & 0x7 != 0`

- ▶ Selects TCP traffic to/from port 80 with SYN, FIN or RST bits set.
- ▶ `tcp[13:1]` refers to a one-byte value at byte offset 13 of the TCP header.

How the Filter Works

- ▶ BPF implements a register-based filter machine.
- ▶ ASCII filter expressions can be compiled into this machine using `pcap_compile`.

[VISUAL ANCHOR: BPF filter expression → compiled filter machine → selected packets]

BPF: Reducing Capture Overhead

- ❶ **Kernel filtering:** filtering in the kernel reduces copying from kernel space to user space.
- ❷ **Snapshot length (snaplen):** only part of each packet needs to be passed to the application.
- ❸ **Buffering:** packets are buffered and delivered when the buffer is full or a read timeout expires.

Examples from the Source

- ▶ tcpdump's cited default snaplen is 96 bytes.
- ▶ tcpdump's cited read timeout is 1000 ms.
- ▶ A RARP daemon's cited timeout is 0 because it needs a prompt response.

[VISUAL ANCHOR: Kernel filter + snaplen + buffer reducing application-side overhead]

Double Buffering

- ▶ BPF maintains two buffers for each application.
- ▶ One buffer can be filled while the other is copied to the application.
- ▶ This is standard **double-buffering**.

Writing to BPF

- ▶ Most applications only read from BPF.
- ▶ Applications can write to BPF to send packets on the datalink.
- ▶ The chapter notes that IP datagrams can instead be created using the `IP_HDRINCL` socket option.
- ▶ Writing to BPF is useful for packets that are not IP datagrams, such as RARP replies.

[VISUAL ANCHOR: Two BPF buffers alternating between capture and application processing]

29.3 Datalink Provider Interface (DLPI)

Definition

DLPI is a protocol-independent interface designed by AT&T for the datalink service provided by SVR4.

Access Styles

- ▶ One style uses a single device and specifies the interface with `DL_ATTACH_REQ`.
- ▶ Another style opens the desired device directly, for example `1e0`.

STREAMS Modules

- ▶ **pfmod**: packet filtering within the kernel.
- ▶ **bufmod**: buffers data destined for the application.
- ▶ Both support functions analogous to BPF filtering and buffering.

[VISUAL ANCHOR: Textbook Figure 29.2 – Packet capture using DLPI, pfmod and bufmod]

DLPI vs BPF: Filtering Model

Aspect	BPF	pfmod / DLPI
Filter model	Directed acyclic control-flow graph (CFG)	Boolean expression tree
Natural machine mapping	Register machine	Stack machine
Filtering before copy	Yes, as described	Depending on implementation, packet may be copied before pfmod discards it
Performance statement in source	CFG implementation reported as normally 3–20 times faster, depending on filter complexity	Comparison basis is Boolean expression tree

[VISUAL ANCHOR: CFG/register-machine path versus Boolean-expression-tree/stack-machine path]

29.4 Linux: SOCK_PACKET and PF_PACKET

Two Linux Methods

- ▶ **SOCK_PACKET**: original method; widely available but less flexible.
- ▶ **PF_PACKET**: newer method with more filtering and performance features.
- ▶ Both require sufficient privileges, similar to raw sockets.
- ▶ The third argument to `socket` specifies the Ethernet frame type.

PF_PACKET Packet Types

- ▶ **SOCK_DGRAM**: “cooked” packet; link-layer header removed.
- ▶ **SOCK_RAW**: complete link-layer packet.
- ▶ **SOCK_PACKET** returns the complete link-layer packet.

[VISUAL ANCHOR: Textbook Figure 29.4 context – Linux packet socket access to datalink frames]

Protocol Selection

Examples given by the source:

- ▶ `ETH_P_ALL`: receive frames for all protocols.
- ▶ `ETH_P_IP`: receive IPv4 frames.
- ▶ Other examples include `ETH_P_ARP` and `ETH_P_IPV6`.

Promiscuous Mode

- ▶ If supported by the datalink, promiscuous mode can be requested.
- ▶ Newer systems use `PACKET_ADD_MEMBERSHIP` with `PACKET_MR_PROMISC`.
- ▶ Older systems can use interface flags with `SIOCGIFFLAGS` and `SIOCSIFFLAGS`.

[VISUAL ANCHOR: Protocol selector + promiscuous-mode membership feeding a packet socket]

Buffering / Filtering

- ▶ No BPF-style kernel buffering in the described Linux feature.
- ▶ Newer systems can use `SO_ATTACH_FILTER` for kernel filtering.
- ▶ Multiple frames cannot be grouped into one read in the described mechanism.

Device Selection

- ▶ `SOCK_PACKET` does not filter by device; `PF_PACKET` can use `bind`.
- ▶ `ETH_P_IP` may deliver IPv4 packets from multiple devices.
- ▶ The application may need to discard unwanted-device traffic.

[VISUAL ANCHOR: Linux capture path showing extra packet copying and device-selection issue]

29.5 libpcap: Portable Packet Capture

Purpose

libpcap provides implementation-independent access to the packet-capture facility supplied by the operating system.

- ▶ Supports packet capture mechanisms including BPF, DLPI and Linux SOCK_PACKET/PF_PACKET in the source's discussion.
- ▶ Used by **tcpdump**.
- ▶ The library functions use the `pcap_` prefix.
- ▶ The source describes about 25 functions and develops a complete example using common functions.

Applications use a common packet-capture interface without depending directly on the OS-specific datalink mechanism.

[VISUAL ANCHOR: Textbook Figure 29.17 – libpcap separating application code from device-dependent capture]

29.6 libnet: Packet Creation and Injection

Purpose

libnet provides an implementation-independent interface for crafting and injecting arbitrary packets.

What it Hides

- ▶ Details of constructing IP and UDP/TCP headers.
- ▶ Some raw-socket and datalink differences.
- ▶ Portability issues such as checksum handling and IP header byte order.

Library Interface

- ▶ Functions use the `libnet_` prefix.
- ▶ The source's example uses the 1.1 API.
- ▶ libnet can output through raw-socket and datalink access modes.

[VISUAL ANCHOR: libnet hiding OS-specific packet-construction details]

29.7 UDP Checksum Case Study: The Question

Goal

Build a program that:

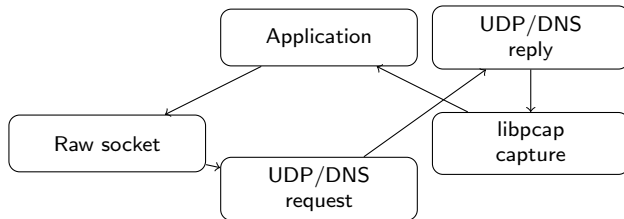
- 1 Creates a UDP datagram containing a DNS query.
- 2 Sends the query to a name server using a raw socket.
- 3 Captures the name server's reply using libpcap.
- 4 Examines the UDP checksum field.

Why This Approach?

- ▶ With IPv4, UDP checksum computation is optional in the source's discussion.
- ▶ A normal UDP socket cannot expose the complete received datagram in the required way.
- ▶ A raw socket is used for packet creation, while packet capture is used to obtain the complete reply.

[VISUAL ANCHOR: Textbook Figure 29.3 – Application using raw socket for request and libpcap for reply]

Case Study: Overall Data Path



Important Observation

The program writes its own UDP datagram through a raw socket and reads the server reply through libpcap.

[VISUAL ANCHOR: Textbook Figure 29.3 – Complete case-study data flow]

Program Structure: Main Functions

The source summarizes the program using these major functions:

Function	Role
open_pcap	Open and initialize packet-capture device
send_dns_query	Build and send the DNS query
udp_write	Build UDP/IP headers and send raw IP datagram
udp_read	Read reply from packet-capture device
test_udp	Send query, wait for reply, inspect checksum
next_pcap	Obtain next captured packet

[VISUAL ANCHOR: Textbook Figure 29.4 – Program function structure]

Options Used in the Source

-0: no UDP checksum -i: capture interface -1: local IP/port -v: verbose output

Why the Local IP Address Matters

- ▶ The source IP is required for the UDP pseudoheader and checksum calculation.
- ▶ If -1 is absent, a UDP socket connection is used to determine the appropriate local address; the destination host and service remain command-line arguments.

[VISUAL ANCHOR: Command-line options feeding destination resolution and local-address selection]

Opening the Packet Capture Device: `open_pcap`

- ❶ If no interface is specified, `pcap_lookupdev` selects a device.
- ❷ `pcap_open_live` opens a live capture device and receives:
 - device name,
 - snaplen,
 - promiscuous flag,
 - read timeout,
 - error buffer.
- ❸ `pcap_lookupnet` obtains network address and subnet mask.
- ❹ The filter expression is constructed from the destination host and service.
- ❺ `pcap_compile` compiles the filter.
- ❻ `pcap_setfilter` loads the filter into the capture device.
- ❼ `pcap_datalink` identifies the datalink type.

Why datalink type matters

The receiver needs to know the size of the datalink header before interpreting the IP/UDP portion.

[VISUAL ANCHOR: Textbook Figure 29.9 – `open_pcap` processing pipeline]

Execution Sequence

Establish SIGALRM handling and a jump buffer; send the DNS query; increment the request count; set the alarm; read with `udp_read`; cancel the alarm after a reply; inspect `ui_sum`.

Timeout Handling

- ▶ Starts at 3 s.
- ▶ Doubles to 6 s, then 12 s.
- ▶ Stops after three unsuccessful requests.
- ▶ Called **exponential backoff** in the source.

[VISUAL ANCHOR: Textbook Figure 29.10 – test_udp and Figure 29.11 – sig_alarm]

DNS Query Data

The raw-socket version builds the DNS query in application memory:

- ▶ Identification = 1234.
- ▶ Flags = 0x0100 (recursion desired).
- ▶ One question.
- ▶ Zero answer, authority and additional records in the query.
- ▶ Query name: `a.root-servers.net.`
- ▶ Query type = A.
- ▶ Query class = 1 (IP address).

Writing the Query

The source computes the user-data length and passes the buffer to `udp_write`, which constructs the UDP/IP datagram.

[VISUAL ANCHOR: Textbook Figure 29.12 – `send_dns_query` raw-socket DNS construction]

Raw-Socket Preparation

- ▶ Create a raw socket.
- ▶ Enable `IP_HDRINCL`, allowing the application to write the complete IP datagram including the IP header.
- ▶ Superuser privileges are required in the source's example.

UDP Checksum

- ▶ The checksum includes UDP header/data plus fields from the IP header forming the **pseudoheader**.
- ▶ The pseudoheader provides additional verification that the datagram reached the correct host and protocol.
- ▶ If the calculated checksum is 0, the source stores `0xffff`; UDP uses zero to indicate that no checksum was supplied.

[VISUAL ANCHOR: Textbook Figure 29.13 and Figure 29.14 – raw socket and `udp_write`]

Packet-Capture Read

- ▶ `next_pcap` calls `pcap_next`.
- ▶ If no packet is ready because of timeout, it loops and calls `pcap_next` again.
- ▶ The captured length is obtained from `pcap_pkthdr.caplen`.

Datalink-Dependent Offsets

- ▶ Loopback (`DLT_NULL`): 4-byte header in the source's example.
- ▶ Ethernet (`DLT_EN10MB`): 14-byte header.
- ▶ SLIP and PPP: 24-byte offsets in the cited BSD/OS example.
- ▶ The program branches using the value returned by `pcap_datalink`.

[VISUAL ANCHOR: Textbook Figures 29.15 and 29.16 – `udp_read` and `next_pcap`]

Verifying the Captured Packet

udp_check

The source verifies:

- ▶ Packet is long enough for IP and UDP headers.
- ▶ IP version is correct.
- ▶ IP header length is valid.
- ▶ IP header checksum is valid.
- ▶ Protocol field identifies UDP.

Capture Statistics

pcap_stats reports:

- ▶ packets received by the filter,
- ▶ packets dropped by the kernel.

If the received UDP checksum field is **0**, the source concludes that the server did not calculate and send a UDP checksum; otherwise, the checksum is present.

[VISUAL ANCHOR: Textbook Figure 29.19 – udp_check and packet-capture verification]

Packet Construction

- ▶ Initialize libnet using `libnet_init`.
- ▶ Build DNS query with `libnet_build_dnsv4`.
- ▶ Build UDP header with `libnet_build_udp`.
- ▶ Build IPv4 header with `libnet_build_ipv4`.

Checksum and Output

- ▶ Passing checksum field as 0 causes libnet to calculate it automatically.
- ▶ Checksum can be disabled with `libnet_toggle_checksum`.
- ▶ `libnet_write` sends the assembled datagram.
- ▶ libnet handles IP-length byte order automatically in the source's comparison.

Source Comparison

The libnet `send_dns_query` version is 67 lines; the combined raw-socket version is 96 lines and contains portability issues discussed by the source.

[VISUAL ANCHOR: Textbook Figures 29.20 and 29.21 – libnet initialization and DNS query construction]

Aspect	Raw sockets	libnet
Packet construction	Application builds headers directly	Library functions build headers
IP_HDRINCL	Used in the source's raw-socket example	Hidden by library interface
Checksum handling	Application calculates/checks it	Library can calculate it automatically
Byte-order portability	Source discusses portability issues	libnet handles IP length byte order in the example
Abstraction	Lower-level control	Higher-level packet construction interface

[VISUAL ANCHOR: Side-by-side raw-socket construction versus libnet abstraction]

29.8 Summary: What Should You Remember?

Datalink Access

- ▶ Applications can observe datalink traffic.
- ▶ Promiscuous mode can expand packet visibility.
- ▶ BPF, DLPI and Linux packet sockets expose OS-specific mechanisms.
- ▶ libpcap provides portable packet capture.
- ▶ libnet provides portable packet creation/injection.

Case Study

- ▶ DNS query is built and sent using a raw socket.
- ▶ Reply is captured using libpcap.
- ▶ Datalink headers are skipped according to capture type.
- ▶ IP/UDP headers are verified.
- ▶ UDP checksum field is examined.

[VISUAL ANCHOR: Chapter 29 summary map]

Scenario

A network-programming application must send a custom UDP/DNS packet and inspect the complete returned datagram.

- 1 Why is datalink access useful instead of relying only on a normal UDP socket?
- 2 Why are raw sockets and libpcap used for different directions of the experiment?
- 3 What is the role of `IP_HDRINCL`?
- 4 Why must the program know the datalink type before locating the IP header?
- 5 What does a received UDP checksum value of zero mean in this case study?
- 6 What problem does libnet solve compared with manually constructing the packet?

[VISUAL ANCHOR: End-to-end case-study challenge: construct → transmit → capture → inspect]